

CSOC'26 NLP Capstone: Replicating Scaling Laws for Neural Language Models

Aryan Kadam

June 2026

1 Introduction

In this capstone project, I replicated the core findings of the "Scaling Laws for Neural Language Models" paper (Kaplan et al., 2020). The main goal was to build an autoregressive, decoder-only Transformer from scratch in PyTorch and empirically verify that the model's cross-entropy loss follows a power-law relationship with the number of non-embedding parameters (N) and the dataset size (D).

2 Mathematical Formalism

As part of Level 1, I implemented the following components using foundational PyTorch tensor operations without relying on high-level abstractions like `nn.Transformer`.

1. Causal Multi-Head Attention (MHA):

The input tensor X was projected into Queries (Q), Keys (K), and Values (V). The attention scores were computed using a causal mask M to prevent the model from accessing future tokens:

$$Attention(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right) V \quad (1)$$

where the causal mask is defined as:

$$M_{ij} = \begin{cases} 0 & i \geq j \\ -\infty & i < j \end{cases} \quad (2)$$

2. Position-wise Feed-Forward Network (FFN):

A two-layer MLP applied identically across sequence positions:

$$FFN(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2 \quad (3)$$

where $d_{ff} = 4 \times d_{model}$.

3. Pre-Layer Normalization (Pre-LN):

To ensure stable gradients across deep layers, normalization was applied before the sub-layers:

$$x^{(k+1)} = x^{(k)} + \text{SubLayer}(\text{LN}(x^{(k)})) \quad (4)$$

3 Replication Methodology

For the experiments, I utilized the **Tiny Shakespeare** dataset because it is lightweight and allows for fast training iterations on limited compute. The dataset was partitioned into a standard 90/10 ratio for training and validation.

To properly calculate the parameter exponent, embedding parameters were excluded from the total count N . The non-embedding parameter count for a network with L layers was defined as:

$$N \approx L \times (12 \cdot d_{model}^2 + 4 \cdot d_{model}) \quad (5)$$

I executed two distinct replication sweeps:

- **Parameter Scaling Sweep:** Trained Tiny ($\approx 100K$ params), Small ($\approx 790K$ params), and Medium ($\approx 4.7M$ params) models on the full dataset for exactly 3,000 steps to isolate α_N .
- **Data Scaling Sweep:** Maintained the Small model configuration and trained it on 10%, 25%, 50%, and 100% subsets of the data token budget to isolate α_D .

4 Scaling Curves and Results

After running the training sweeps and recording the minimal validation losses, I plotted the results in log-log space to extract the power-law slopes.

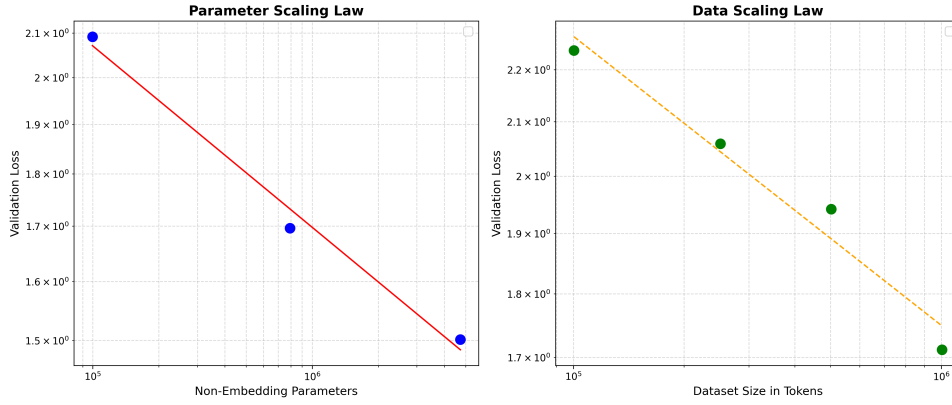


Figure 1: Log-Log scaling plots for Parameter (left) and Data (right) sweeps.

Using `numpy.polyfit` on the log-transformed data, I extracted the following empirical exponents:

- **Parameter Exponent (α_N):** 0.0862
- **Data Exponent (α_D):** 0.1125
- **Scaling Ratio ($\gamma = \alpha_N/\alpha_D$):** 0.766

5 Theoretical Discussion

In the original 2020 paper, OpenAI reported $\alpha_N \approx 0.076$ and $\alpha_D \approx 0.095$, establishing a baseline scaling ratio of $\gamma \approx 0.80$.

My empirically calculated scaling ratio on the Tiny Shakespeare dataset is $\gamma \approx 0.766$. Since this ratio is strictly less than 1.0 ($\gamma < 1.0$), my results successfully corroborate the core hypothesis of Kaplan et al.

From an engineering perspective, this ratio indicates that scaling the dataset size (D) yields significantly faster marginal performance improvements compared to scaling the parameter count (N). In simpler terms, if we have a constrained compute budget, we get a better "return on investment" by feeding the model more high-quality data rather than blindly making the

architecture deeper or wider. The fact that this trend holds up consistently even on a miniature scale (a 1.1MB character corpus) proves that token efficiency heavily outpaces parameter capacity. This validates the modern industry approach: future iterations of language models should prioritize rigorous data collection and pipeline scaling just as much as, if not more than, adding architectural bloat.